

把本体当成内核，给 agent 一份可执行的契约 Toward Effective and Reliable LLM Agents via Dynamic Ontology

本篇范围说明：这是一篇 arXiv 论文 (2608.22974v1, 2026-08-24, 南京大学新型软件技术国家重点实验室等)。下面「全文翻译」一节按章节覆盖摘要、引言（问题与缺口）、相关工作的三条线、OaK 框架的四步构造循环与推理阶段、实验结果、案例研究、成本分析、结论与局限；参考文献列表、附录里的算子签名与提示词模板不译；另外三个基准的完整结果表里有两个抽不出数字（PDF 里是嵌入字体的字形码），只有 CRMArenaPro 那张能读，下面报的是它。

核心内容

Q：这篇论文要解决的问题是什么？

A：大模型 agent 依赖的知识，要么编码在模型参数里、要么以非结构化上下文呈现。在领域特定任务里，这让重要的语义联系保持隐式。

后果论文写得很直接：证据用不全，多步决策脆弱。

Q：那用本体不就行了？

A：这里有两层障碍，而第二层是这篇论文真正的出发点。

第一层是老问题：构造可用于任务的本体传统上需要领域专家投入大量精力，难以规模化。

第二层才是关键，也是我认为这篇最有价值的一句判断：

自动构造同样有挑战：一个看起来语义上说得通的本体，可能并不包含实际决策所需要的关系结构。

换句话说：本体「像那么回事」和本体「能用来做决定」是两件事，而自动构造很容易只做到前者。

Q：论文对「本体」的用法和常规不同，差别在哪？

A：这是理解全篇的关键，论文自己把它挑明了：

然而，常规的本体主要是描述性的：它规定一个领域里存在什么，但不必然规定一个 agent 可以做什么。它也没有回答一个操作应当如何被参数化、以及在其结果被接受之前哪些条件必须成立。

所以他们**在操作性的意义上使用「本体」**：数据表示上的要求编码在 schema 声明里，而计算与 workflow 上的要求由函数的控制流强制执行。

落点是一句很有分量的话：

这样的本体不是一份被动的知识描述。它充当一份语义与程序上的契约，划定 agent 可以做什么的边界，并让执行保持可被检查。

Q：框架叫什么，长什么样？

A：叫 **OaK** (ontology-as-a-kernel，本体即内核)。核心是一个内核 $K = (S, F)$ ：

- **S 是面向任务的 schema**，定义 agent 可用的领域概念、它们的属性与关系
- **F 是一组有类型的推理函数**，定义这些带类型的元素如何通过可执行过程被使用——检索、过滤、遍历、投影、聚合、多步推理

在 S 之下，从任务数据实例化一张 **schema 引导的知识图谱 G**，函数就在这张图上操作。

「动态」指的是什么，论文专门定义了：任务条件下的构造——对每个任务，OaK 自动构造解决它所需的 schema 与有类型的推理函数，再实例化对应的图，refine 之后冻结内核用于推理。

| 四步构造循环

每一轮从训练集里重新随机抽一个样本子集（论文说明了理由：每轮重采样让循环见到多样的数据，防止 schema 与函数过拟合到一个固定子集）。

第一步：schema 构造，三个阶段

需求分析 → 大模型读任务描述与本轮样本，产出一份**需求规格说明**，记录任务范围、关键实体与关系、任务约束。论文强调它的作用是「给 schema 一个精确的靶子，而不是一份自由发挥的简报」。

schema 起草 → 从需求规格加上上一轮的修复反馈起草：先枚举实体类型与属性，再声明它们之间有类型的关系。每个实体类型被指派一个身份字段作为主键，每条关系指明源与目标实体类型。

形式化验证 → 这一步是我认为整篇最该抄的一处设计。

一份起草出来的 schema 可能看起来语义合理、内部却在逻辑上有缺陷，而这种缺陷会传播进图里。因此我们在使用之前验证它。

做法很聪明：他们的 schema 是**轻量的**——一份 YAML 或 JSON 文档，列出命名的实体类型、属性、有类型的关系，agent 直接读它，而不是完整的形式化本体那套公理机制。

这让模型易于消费，但逻辑推理器没法照原样检查它。因此我们把它编码成一份等价的 OWL 本体，并运行 HermiT。

HermiT 检查五类逻辑有效性：**互斥性、限制、属性层面、全局一致性，加上不可满足的类**。任何一项不过，schema 就带着推理器给出的反例被退回起草阶段重来。

第二步：知识图谱实例化

用 **分块—映射—合并** 流水线：语料先切成符合抽取器输入预算的块，一个大模型抽取器按当前 schema 把每块映射成有类型的实体与关系候选，再由合并算子把这些集合调和成一张图。

合并靠的是 schema 声明的主键：每个候选实体被赋予一个**键签名**（实体类型 + 主键值），**两个候选指代同一个现实对象，当且仅当它们的键签名相同**。因为这构成一个等价关系，合并算子把每个等价类塌缩成一个规范节点，关系再重新挂到规范端点上。

这一步解决的是一个具体问题：抽取器在每块上独立运行，所以同一个现实实体可能在多个块里各冒出一个候选。

第三步：知识推理——把通用算子编译成领域函数

通用算子库提供基本的图与数据操作：实体查找、关系遍历、属性投影、约束过滤、聚合。

一个函数把某个任务里反复出现的推理步骤编译成一次带类型的调用。论文给的理由是：

这把 agent 从多步规划中解放出来。模型内部做的推理越少，留给幻觉的空间就越小。

函数以三种方式落到通用算子上：**组合**若干算子成流水线、**特化**某个算子（固定或重新解释其参数）、或**适配**某个算子（加轻量的前后处理）。每个函数都在当前图上测试过可执行，通过验证的目录才作为可调用工具暴露给 agent。

第四步：本体评估——judge 驱动的修复

答案先按数据集自己的评测协议打分，**然后一个 judge 模型审阅整个内核连同轨迹与分数**。

修复建议的结构值得记：每条建议是一个四元组（目标产物，动作，提议的定义或补丁，诊断出的原因），动作取自「新增／删除／修改」。整份报告按目标切分成 **schema 层的**（喂给下一轮第一步）和**函数层的**（喂给下一轮第三步）。

循环停止条件：judge 找不到阻塞性缺陷，或迭代预算用尽。

推理阶段：冻结，然后只在内核里跑

循环结束后冻结最终一轮的 schema 与函数。对测试查询，先建一张推理用的图，然后 ReAct agent 用冻结的内核求解。

与自由形式的 agent 推理不同，这里的推理被路由**穿过一个经任务 refine 过的内核**：schema 关掉了 agent 可以援引的概念与关系，函数把它的计算限制在受 schema 约束的图实例上、有类型的可执行组合。因此 agent 在一个经过验证的语义空间内部操作。

实验结果

三个基准：**TravelPlanner**（旅行规划）、**CRMArenaPro**（企业 CRM 任务）、**ToolQA**（工具问答）。两个底座：DeepSeek-v4-flash 与 GPT-4o-mini。

CRMArenaPro 这张表能读出数字（DeepSeek-v4-flash 底座）：

方法	B2B 平均	B2C 平均	B2B 政策	B2B 数据库
ReAct	60.78	60.87	47.50	71.56
AFlow	56.20	54.21	46.25	48.44
MemP	66.44	67.27	55.62	78.12
ReCode	24.96	24.41	39.38	24.06
AgentSquare	55.98	56.36	46.88	50.94
OaK	78.38	75.20	81.88	94.69

优势最明显的是政策与数据库两类任务（政策 81.88 对次优的 55.62；数据库 94.69 对 78.12）——论文的解释是这两类要求 agent 同时做到业务规则合规与准确访问记录。

TravelPlanner 那边有一条定性观察值得记，因为它说明了这个方法针对的是什么：面向工作流的基线在单项约束上表现不错，但满足单个约束不必然产出一个联合有效的计划。旅行规划要求日期、城市、交通、住宿、餐饮、预算在多天之间保持相容，跨部件的协调才是关键。

构造循环收敛得很快：三个基准上前几轮快速提升，第 4 - 5 轮之后基本走平，说明五轮预算捕获了迭代 refine 的大部分收益。

成本上有一个反直觉的结果：OaK 取得最高通过率的同时，输入 token 比 ReAct 和 MemP 更少（论文的解释是 schema 适配的函数减少了从大段上下文里重建图操作的需要）；但运行时间与输出 token 更高，主因是它在规划之前要实例化一张查询专属的图。当一张图能在相关查询之间复用时，这笔构造成本可以摊薄。

一个具体的修复案例，值得完整看

这是全篇最能说明「schema 与函数必须一起修」的地方。

schema 修复：早期某一轮里，judge 发现「酒店」实体暴露了价格、房型这类可见属性，**却漏掉了「最少入住晚数」与「最大入住人数」两个约束字段**。没有这两个字段，agent 可能选中一家看起来便宜又合适、实际却因为要求更长住宿或住不下全部人而无效的酒店。judge 于是发出一条「新增」建议。

函数修复：但只改 schema 不够——取候选住宿的那个函数还没有用上新暴露的约束。本体评估器追踪剩余的失败计划，发出一条「修改」建议，**给函数加上按人数和最少晚数的过滤**。修复后，只有当最大入住人数覆盖同行人数、且最少入住晚数适配计划住期时，那家酒店才会被保留。

这个案例的形状是：字段加了不等于用了。而且两步是由同一个 judge 在两个不同层面分别诊断出来的。

背景补充 / 点评

先把三个词落到地面。

ReAct 是把推理与行动交错在同一条执行轨迹里的经典范式——想一步、调一次工具、看结果、再想。这篇的 agent 用的就是 ReAct，变的不是 agent 循环，是它能调用的东西。

HermiT 是一个 OWL 本体的逻辑推理器，能检查本体内部是否自治、有没有「不可满足的类」（定义上永远不可能有实例的类，通常意味着建模出错）。

「不可满足的类」这个检查项特别值得注意，因为它抓的是一类静默错误：一个类的约束互相矛盾，导致它永远为空，而系统不会报错、只会永远查不到东西。

再说这篇对手上项目的用处，先说 **prove**，因为对得最准。

prove 的形态是把用户问题编译成对数据仓库的查询并作答。按 8/22 读的那篇 Agentic SQL 的自主度分档，它是 **L2**（多次调用沿固定流水线，无动态路由）。

OaK 提供的是一条不同于「往 L3 走」的路：不是让 agent 更自由地分支，而是给它一个更受约束但可执行的接口。

具体能借的有三条。

第一条，也是最该借的：把 schema 编码成 OWL 跑一次逻辑检查。

论文的做法是——日常使用的 schema 是轻量的 YAML/JSON（模型好读），但验证时把它编码成等价的 **OWL 交给 HermiT**。判据一句话：「一份 schema 可能看起来语义合理、内部却在逻辑上有缺陷，而这种缺陷会传播进下游」。

这一条对学员画像项目直接适用。那边现在有五个 field、各自的受控词表、刻度、以及 8/24 定下的判重键（`learner_id, field, predicate, scale, condition`）。这些约束目前是靠人读文档保证一致的，没有任何机械检查。而 OaK 那五类检查里至少有两类能直接对上：**属性层面的一致性**（同一个 field 下的派生属性是否单一声明——这正是 8/24 加的那条加载期不变量），以及**不可满足的类**（有没有哪个 field 加词表加刻度的组合永远不可能有实例）。

但要说清代价：把画像的 schema 编成 OWL 需要额外工作，而画像现在只有几十条记录、五个 field。这件事的收益随 schema 复杂度增长，现在还不到做的时候。值得记的是那个判据本身——**当 schema 复杂到人读不出矛盾时，就该上机械检查了。**

第二条，那句「模型内部做的推理越少，留给幻觉的空间就越小」，是一条可操作的设计原则。

OaK 的做法是**把反复出现的推理步骤编译成一次带类型的调用**，而不是让 agent 每次现场用通用算子拼。这和 8/25 读的那篇「让答案生来可审计」里的 CARP 是同一个方向——都是把本来发生在模型脑子里的决定，搬到一个可以被检查的结构里。

prove 那边已经部分做到了（指标口径写在配置里、编译时展开），但「反复出现的推理步骤」这一层还没有对应物。比如「同比」「环比」「占比」这类派生计算，如果每次都靠模型现场组合，那就是论文说的「留给幻觉的空间」。

第三条，judge 修复建议那个四元组结构值得抄。

（目标产物，动作，补丁，诊断原因）——尤其是最后那个「诊断原因」。只给补丁不给原因，下一轮无法判断这次修复是否对症；而按目标切分成 schema 层与函数层，让两类反馈各自回到对应的构造步骤，避免了「知道哪里坏了但不知道该改哪一层」。

而那个酒店案例说明了为什么必须分两层：字段加了不等于用了，schema 修复与函数修复是两次独立的诊断。

第四处，那个「重采样防过拟合」的细节，是一条容易被忽略的实验纪律。

每轮从训练集重新随机抽样，**防止 schema 与函数过拟合到一个固定子集**。这和 8/23 读的那篇 agent 项目记忆论文里的「标准答案从一手来源撰写、绝不从图里来」是同一类纪律——**都是防止用来改进系统的东西，反过来被系统塑造。**

最后说这篇该保持怀疑的地方。

第一，论文自己列的三条依赖都是硬依赖。 结论里写明：OaK 仍要付图实例化的成本；它的质量取决于基于大模型的抽取器与 judge；并且需要一个可靠的任务评估器。

第三条依赖最要命，而论文没有展开。 整个构造循环的驱动力是「按数据集的评测协议打分」——这三个基准都自带官方评分器。而真实业务任务通常没有。没有可靠评估器，judge 就没有客观信号可依，整个 refine 循环退化成模型自己评自己。这个前提在学术基准上唾手可得，在企业场景里往往是最难的那一环。

第二，两个基准的结果表我抽不出数字。 TravelPlanner 与 ToolQA 那两张表在 PDF 里是嵌入字体的字形码，所以那两个基准上的提升幅度我只能转述论文的定性说法（「最高通过率」「大多数子集第一」），拿不到具体数值去判断幅度。CRMArenaPro 那张能读，提升幅度确实很大（B2B 平均 78.38 对次优 66.44）。

第三，成本那一节的结论有条件。 「输入 token 更少」成立，但「运行时间与输出 token 更高」也成立，而论文给的缓解是「当一张图能在相关查询之间复用时，这笔构造成本可以摊薄」——这是一个条件句，而实验里每个查询都建一张专属图，也就是说这个摊薄在实验里并没有发生。

第四，「五轮走平」这个观察可能是预算决定的、不是收敛决定的。论文说第 4 - 5 轮之后曲线基本走平、五轮预算捕获了大部分收益，但实验本身就是五轮预算。走平和预算耗尽在这张图上分不开。

一句话版本：这篇把本体从「描述领域里有什么」改造成「规定 agent 可以做什么」——一份带类型函数的可执行契约；最该抄的一处是「轻量 schema 给模型读、编码成 OWL 给推理器查」这个双表示设计，因为它抓的是「看起来合理但内部逻辑有缺陷、且会传播到下游」这类静默错误；而它最大的未言明前提是整个 refine 循环依赖一个可靠的任务评估器，而真实业务任务通常没有。

文中金句

"In domain-specific tasks, this leaves important semantic connections implicit. This often results in incomplete evidence use and brittle multi-step decisions." 在领域特定任务里，这让重要的语义联系保持隐式。结果常常是证据用不全、多步决策脆弱。

"an ontology that appears semantically plausible may not contain the relational structures needed for actual decision making." 一个看起来语义上说得通的本体，可能并不包含实际决策所需要的关系结构。

"a conventional ontology is primarily descriptive: it specifies what exists in a domain, but not necessarily what an agent may do." 常规的本体主要是描述性的：它规定一个领域里存在什么，但不必然规定一个 agent 可以做什么。

"Such an ontology is not a passive knowledge description. It serves as a semantic and procedural contract that bounds what the agent may do and keeps execution open to inspection." 这样的本体不是一份被动的知识描述。它充当一份语义与程序上的契约，划定 agent 可以做什么的边界，并让执行保持可被检查。

"A drafted schema can look semantically reasonable yet be logically flawed inside, and such a flaw would propagate into the graph." 一份起草出来的 schema 可能看起来语义合理、内部却在逻辑上有缺陷，而这种缺陷会传播进图里。

"This makes it easy for the model to consume, but a logical reasoner cannot check it as is. We therefore encode it as an equivalent OWL ontology and run HermiT." 这让模型易于消费，

但逻辑推理器没法照原样检查它。因此我们把它编码成一份等价的 OWL 本体并运行 HermiT。

"A function compiles a task's recurring reasoning steps into a single typed call. This frees the agent from multi-step planning. With less reasoning done inside the model, there is less room for hallucination." 一个函数把任务中反复出现的推理步骤编译成一次带类型的调用。这把 agent 从多步规划中解放出来。模型内部做的推理越少，留给幻觉的空间就越小。

"Resampling per round exposes the loop to varied data and keeps the schema and functions from overfitting to a fixed subset." 每轮重采样让循环见到多样的数据，防止 schema 与函数过拟合到一个固定子集。

"S* closes off concepts and relations the agent may invoke, and F* restricts its computations to typed, executable compositions over schema-constrained graph instances." schema 关掉了 agent 可以援引的概念与关系，函数把它的计算限制在受 schema 约束的图实例上、有类型的可执行组合。

"satisfying individual constraints does not necessarily produce a jointly valid plan." 满足单个约束，不必然产生一个联合有效的计划。

全文翻译

摘要

大模型 agent 严重依赖编码在模型参数中、或以非结构化上下文呈现的知识。（问题、本体的两层障碍、以及 OaK 的定位见「核心内容」。）

给定任务需求与训练数据，OaK 构造一份本体及其知识图谱，生成用于图推理的任务适配函数，并用 judge 反馈迭代地 refine 两者。通过让相关的概念与关系变得显式，这份本体为知识检索与多步决策提供了落脚点。

在 TravelPlanner、CRMArenaPro、ToolQA 上评测，结果显示 OaK 改进了标准的大模型 agent、强化了证据落地、并提升了多步推理的可靠性。

一、引言

论文对 agent 现状的概括是：一个典型的 agent 跑一个循环——反复解读目标并规划，然后通过工具调用或检索行动、观察结果，直到任务完成。近期架构进一步引入了反思、程序性记忆、模块化工作流优化等机制。

它把中心挑战重新表述了一遍，这句是全篇的立论点：

尽管有这些进展，**核心挑战已不再只是 agent 能不能行动，而是当执行变得更长、更自主时，它的行为能否保持可控与可信。**

理由展开得很具体：**在每一个中间步骤，agent 都要决定检索什么、调用哪个工具、传哪些参数，所以这些选择中任何一个出错都会传播到后续步骤。** 因此——

仅有最终答案的准确率，既不揭示支撑证据，也不揭示工具调用的正当性。它还掩盖了记忆的影响与执行失败的来源。

论文随后点出已有手段的共同不足：反思、程序性记忆、工作流优化能改进规划或复用，**但它们通常让「可接受的概念集合与动作序列」保持隐式。**而提示词指令与工具描述引导 agent 的行为，却没有为「它可以执行什么、其结果应当如何被检查」提供一份可强制执行的契约。

由此推出那个缺失的层：**介于大模型与它所控制的工具之间、让领域语义与可执行行为都变得显式的、面向任务的表示。**

四条贡献：提出「本体即内核」；开发一条自动化流水线（构造经验证的 schema、实例化其知识图谱、把通用算子编译成可执行的领域函数）；提出 **judge 驱动的 refine**（用官方任务分数与执行轨迹跨轮诊断并修复 schema 与函数的失败）；在三个基准两个底座上验证。

二、相关工作

三条线，论文对每条都点明了自己的差别。

大模型用于 agent：ReAct 确立了推理与行动交错的范式；MemP 把成功轨迹蒸馏成可复用的程序性记忆；ReCode 用递归生成的程序表示规划与行动；AFlow 与 AgentSquare 自动化 agent 设计。**这些方法逐步把 agent 控制从模型参数里挪到反馈循环与可复用过程中，主要改进的是 agent 如何组织与适配它的控制过程；而 OaK 补的是另一侧——通过一份显式的领域 schema 与可执行的推理函数，去结构化任务接口本身。**

可靠的 agent：Self-Refine 与 Reflexion 用模型生成的批评或执行结果修订响应；ToolEmu 与 AgentDojo 分别揭示高风险工具场景中的现实失败、以及经由不可信观测的间接提示词注入；AgentSpec 把安全需求表达成结构化规则。**这些方法提供反馈、策略强制或执行记录，但通常把任务规则与可允许的操作当作固定输入，或让它们散落在提示词与实现代码里。结果是任务需求与 agent 执行之间的接口难以检查、难以适配。**

图用于 agent: GraphRAG 与 G-Retriever 用图结构检索全局相关或关系性的证据。这些方法主要把图当作外部知识与检索层。这能改进证据获取，但它本身并不规定 agent 可复用的任务级计算。

三、Oak 框架

(内核定义、四步构造循环、推理阶段，见「核心内容」。)

四、实验

(三个基准的结果、消融、构造循环分析、案例研究、成本分析见「核心内容」。)

消融比较了三个变体，其中**去掉函数组合**这一项把通用算子直接暴露给 agent。论文称消融确认了**函数模块、函数组合、以及迭代 refine** 三者的重要性。

五、结论与未来工作

Oak 把领域语义变成大模型 agent 的一个**可执行接口**。这些结果表明，让领域语义与推理过程变得显式，能同时改进大模型 agent 的有效性与可检查性。

局限三条: 仍要付图实例化的成本；质量取决于基于大模型的抽取器与 judge；需要一个可靠的任务评估器。未来工作可研究可复用、增量更新的图来摊薄构造成本，以及更强的验证。

参考链接

- 论文 [arXiv 2608.22974](#) — 2026-08-24
- [ReAct](#) — 推理与行动交错的范式，本文 agent 循环的基础
- [HermiT OWL 推理器](#) — schema 形式化验证用的推理器
- [TravelPlanner](#) — 旅行规划基准
- [ToolQA](#) — 工具问答基准
- [GraphRAG](#) — 把图当外部检索层那一派，本文的主要对照立场
- [AgentDojo](#) — 揭示工具输出可经提示词注入操纵 agent 行为